Scenarios

Scenarios

- A **scenario** illustrates the interaction of a user with a system (...possibly the interaction of multiple types of users)
  - User stories (from Agile) and storyboards are types of scenarios, among others
- A scenario is used as a tool to illustrate and document requirements, but it's also used to elicit requirements as a discussion point
- i.e. does this scenario capture a shared understanding of the system between the user and developers?

Scenarios

- Scenarios have a **goal**, i.e. what is to be achieved
- Scenarios have a **timeframe**, i.e. a meal at a restaurant
- Scenarios have a **scope**, i.e. the entire system or just part of the system
- These may be implicit in the scenario, instead of explicitly noted

Scenarios

- Scenarios can be used as a tool in requirements gathering
  - Draft scenarios can be created to represent an understanding of stakeholder needs
  - Stakeholder needs can be verified against these scenarios, and scenarios can be modified in response to feedback
- Scenarios are very often used as requirements specifications

User Stories

- A **user story** is a natural language description of the features of a system written from the perspective of an end user
  - A user story can be relatively informal, i.e. metrics and technical language are not needed as part of a user story
  - Informality allows all stakeholders to discuss contents

- User stories originated from work by Kent Beck in the late 1990s, formalized in Extreme Programming Explained book (1999)

User Stories

- User stories may be written by the…
  - End user
  - Customer
  - Developer
  - Manager
  - Stakeholders
- Or any combination of the above! Perspective of the user story should be an end user though.
- User stories sometimes written on post-it notes, index cards or whiteboards, but are also frequently digital

User Stories

- Within scrum and other agile development methodologies, a user story represents a **unit of work** to be done
- Product owner in scrum may organize user stories in a product backlog of work to be done
- In some teams, user stories replace traditional requirements completely
- In other teams, user stories are part of specifying requirements, but are also a step towards defining more precise functional requirements

Three C's

- Three aspects to user stories, i.e. the three 3 C's
- **Card** – the text of the user story
  - Called card because it may be a physical index card
- **Conversation** – a conversation about the user story between stakeholders, most often verbal, to further understand the user story
- **Confirmation** – the tests necessary to ensure the objectives of the user story are satisfied

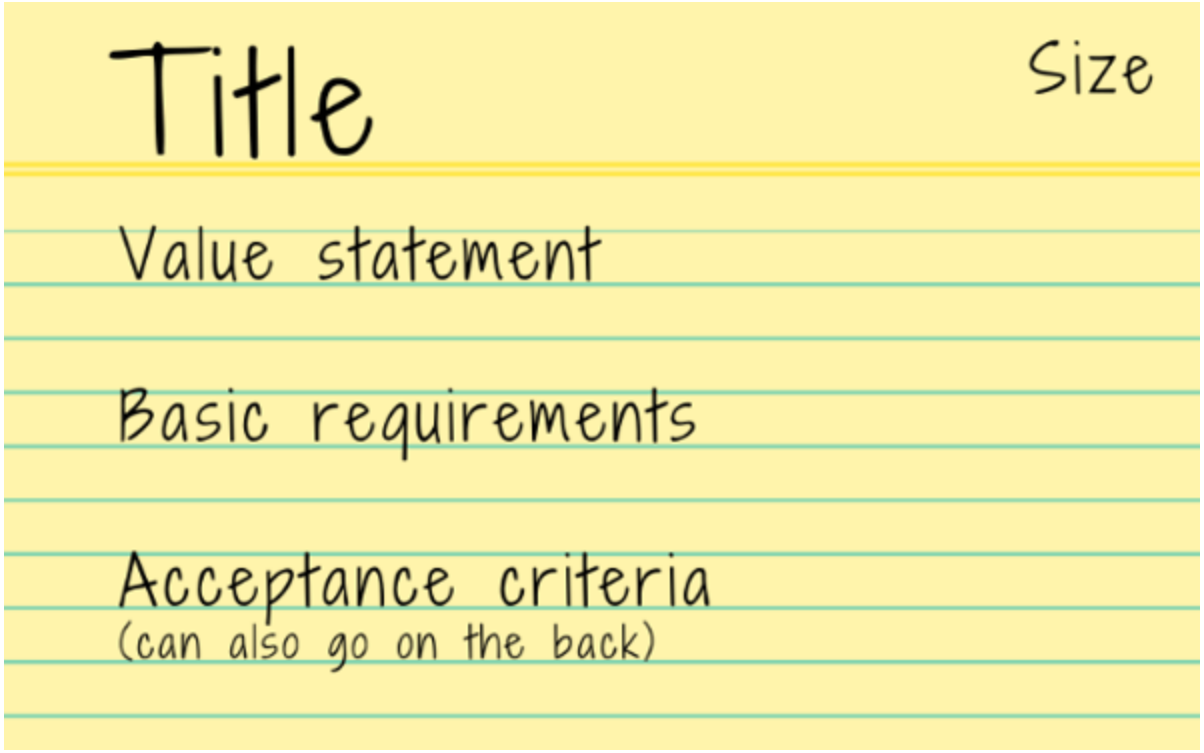
Three C's

- A card will typically be created first based on user needs
- A conversation will take place to better understand the card, i.e. flesh out the details, the more precise requirements for the user story
  - Formality of the documentation of these details varies, and the card itself may be refined as part of this
- Card text is not a requirements specifications in and of itself, it notes that a conversation must occur to fulfill some need, and the conversation fleshes out the requirements
  - Moving from problem to solution

Three C's

- Conversation also includes discussion on what is required to verify deliverable
  - i.e. what are the acceptance criteria
- This would typically involve something like a set of unit tests being passed
- Work is considered complete when tests and any other work required for confirmation are complete

An example card, the value statement is the text of the user story. Basic requirements and acceptance criteria would come after a conversation. Size would be an estimation of the amount of work. And title is just something short to call it. :-)



Source: https://www.leandog.com/blog/how-to-write-the-best-user-stories-with-story-cards

Another template. The acceptance criteria could go on the back of the card.

<Title>	#<ID>
As A:	
I Want To:	
So That:	
CONVERSATION	
•	
•	
•	
•	

Source: https://www.jamieclouting.co.uk/wp-content/uploads/2012/03/card-front-1024x768.jpg

Developing Cards

- Cards are typically developed using a ***template***, i.e. a pattern is followed with who/what/why info

- Example: As a <role> I can <capability>, so that <receive benefit>
  - The "so that" clause is optional but explains the value, "the why", of the work to be done which can be illuminating
  - Template tells us who, what and why in that order

- Example: In order to <receive benefit> as a <role>, I can <goal/desire>
  - This example emphasizes the value of the work to be done first

Card Text Examples

- **Example:** As a teacher I can create multiple choice quizzes, so that I can test my student's understanding.

- **Example:** As a customer I can cancel my order at anytime, so that I do not lose any money if an incident occurs.

- Notice how the benefit/why in both cases might influence the conversation and what is built?

User Story Benefits

- User stories are widely adopted in industry, some developers genuinely believe they are *how things are done the right way*

- And they might be right, but there is no strong empirical evidence demonstrating user stories are objectively advantageous over other approaches
  - Would you let a remotely controlled robot that was built based off user stories to perform surgery on you?
  - We might need to scale our tools to the rigor required!

- But why do they work well and why are they such a popular approach?

User Story Benefits

- A conversation between stakeholders for each distinct unit of work reduces chances for misunderstanding... everyone gets on the same page
  - We say the shared understanding of the work is improved
- Improving shared understanding is key to not just software requirements, but good software architecture (i.e. the overall design of the software)

User Story Benefits

- A verbal conversation to establish a shared understanding can actually be **more precise** than written requirements
  - Natural language is imprecise, details and precision are lost in the process of making requirements
  - Would you like soup or salad? Would you like cream or sugar? One is an exclusive or, the other is not. Context of ordering at a restaurant is the same!
- User stories tend to be a relatively managable chunk of work to be discussed as well, as opposed to trying to come up with requirements for the entire system at once

User Story Benefits

- User stories prioritize the user and the benefit they desire, facilitating better decision making

- If we list the requirements for a car we might have:
  - Runs on gasoline
  - Accelerator pedal to increase speed
  - Trunk to store items

- Do we build a sports car? A minivan?

- But if we know the user benefit is that they want to *move around larger objects like chairs*, we're much more likely to build a van or truck of some kind

User Story Benefits

- Because each user story is a smaller and more manageable chunk of work to be done, estimation becomes easier

- Conversation fleshes out detailed requirements for the story, but there will only be so many

- We can then assign an estimate based on the conversation and shared understanding

User Story Drawbacks

- Do not scale up well to very large problems
- Focus on requirements from perspective of the user may neglect technical requirements to be able to fulfill these user requirements
- What about non-functional requirements?
  - Not impossible to fit them in, but they clearly aren't the focus of a card
- Verbal conversations might develop a shared understanding, but they aren't written contracts
  - What if there is a legal dispute over the work done?

User Story Groupings

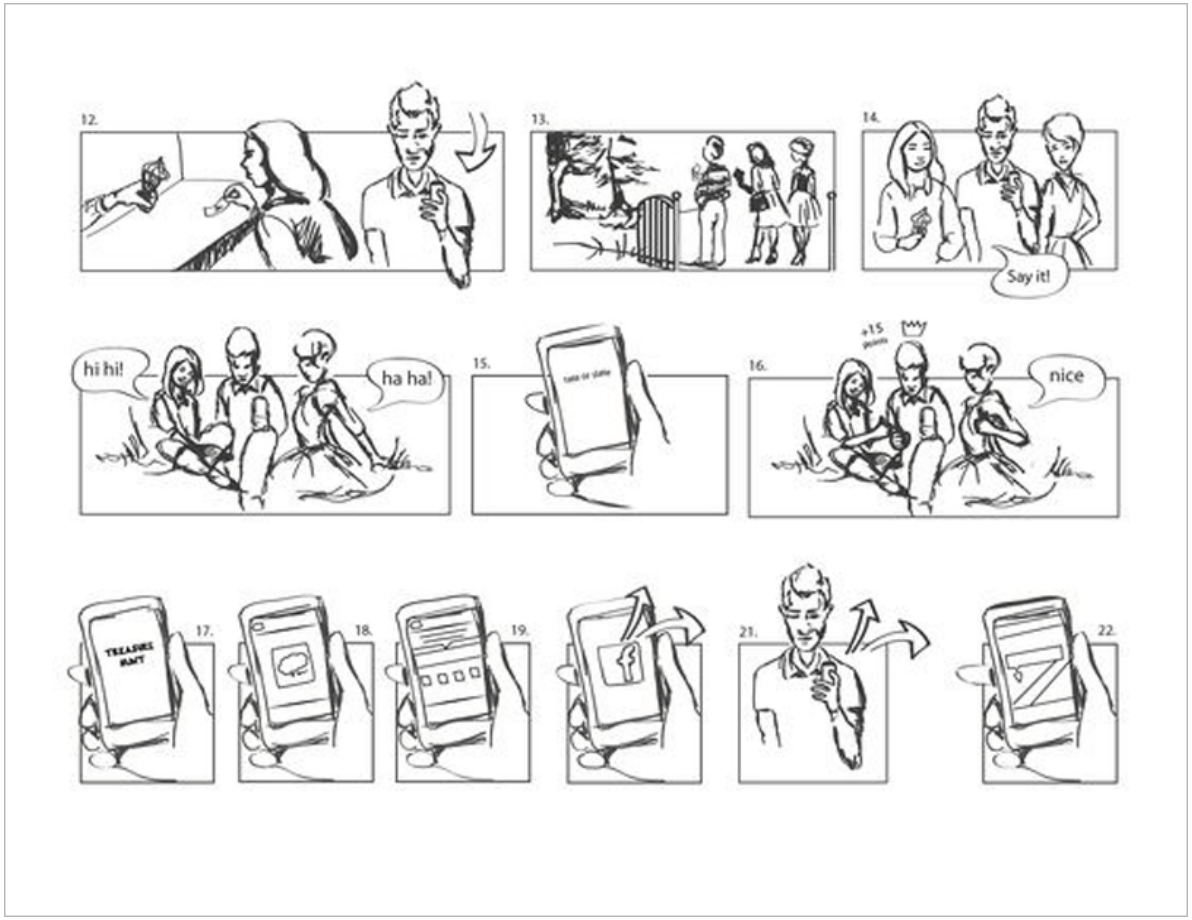
- Groupings and hierarchies of user stories are sometimes created
- The terms epics, themes and initiatives are sometimes used to describing groupings
- A theme may group together multiple user stories, and in turn an epic may group together multiple themes

Other types of scenario representations…

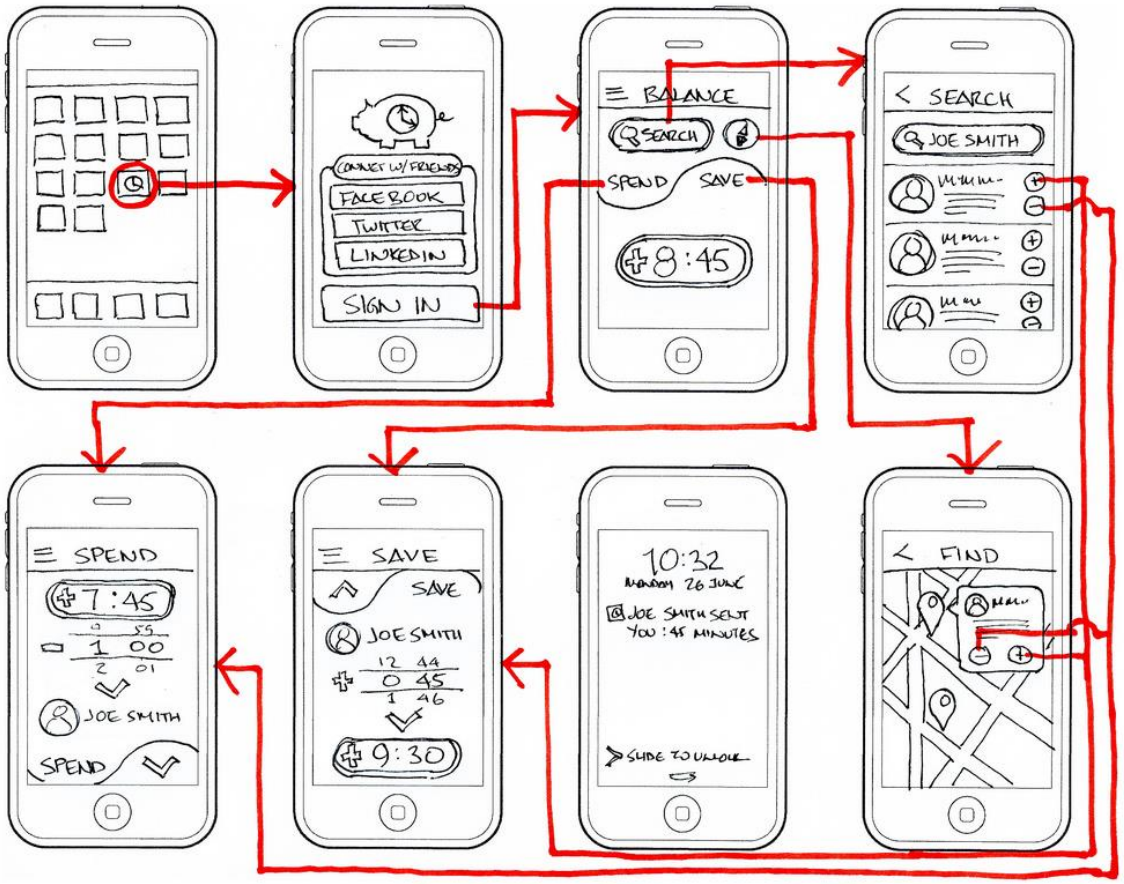
Storyboards

- A storyboard is a sequence of illustrations of a user's interaction with software
- A storyboard can be focused on the user's experience with the software
- A storyboard can be focused on the sequence of interfaces encountered during some process
  - e.g. like a series of prototypes

Storyboard Example



Storyboard Example



Source: https://anniehaydesign.weebly.com/uploads/9/5/4/6/95469676/image01_orig.png

Use Cases

- A **use case** is a representation of a scenario which may include:
  - Title
  - A goal
  - A scope (level)
  - Actors (types of users)
    - A primary acted may be distinguised from the others
  - A list of steps and possibly extensions that may be made up of sublists of steps
    - A list of the main flow of the steps of the application
    - Sublists covering alternative paths, like exceptions or optional paths of steps

Use Cases

- Continuation of what a use case may include....
  - Parts of the overall system involved in the use case
  - Precondition(s), postcondition(s)
  - Trigger
    - What causes the use case to occur?
  - Stakeholders
    - Who is the use case relevant for?
  - Exceptions
    - What can go wrong?
  - Use case diagram (a modelling technique)
  - Other categories may be included too!

Use Case Template Example

USE CASE #	< the name is the goal as a short active verb phrase>	
Goal in Context	<a longer statement of the goal in context if needed>	
Scope & Level	<what system is being considered black box under design> <one of: Summary, Primary Task, Sub-function>	
Preconditions	<what we expect is already the state of the world>	
Success End Condition	<the state of the world upon successful completion>	
Failed End Condition	<the state of the world if goal abandoned>	
Primary, Secondary Actors	<a role name or description for the primary actor>. <other systems relied upon to accomplish use case>	
Trigger	<the action upon the system that starts the use case>	
Description	**Step**	**Action**
	1	<put here the steps of the scenario from trigger to goal delivery, and any cleanup after>
	2	<...>
	3	
Extensions	**Step**	**Branching Action**
	1a	<condition causing branching> : <action or name of sub-use case>
Sub-Variations		**Branching Action**
	1	<list of variations>

Source: https://pja.mykhi.org/0sem/INN/sorcersoft.org/io/uml/images/UML-template.png

Use Cases

- The exact relationship between scenarios and use cases is described differently by different authoritative sources

- Exact rigour and formality of use cases can vary
  - Idea of a *fully dressed use case* vs *casual use case*
  - A fully dressed use case includes all details, a casual use case may only include title/goal, primary actor, scope, level and story/description
  - Generally though, the term use case is used to describe more formal and detailed documentation of a scenario than other methods

- We can use Use Case UML diagrams to help illustrate use cases

Use Case Example

Use Case: Order Product

Goal: Customer places an order for a specific product.
Level: User-goal
Primary Actor: Customer
Pre-conditions: The primary actor is logged into the system

Main Success Scenario:

1. Customer actor indicates that he/she wants to search for a specific product.
2. Customer selects the product category and optionally the desired brand and model.
3. System displays search results that match the customer supplied criteria.
4. Customer selects a specific product and then specifies the desired quantity.
5. System confirms availability of the product (in the requested quantity) and displays the purchase summary.
6. Customer selects the method of payment and enters the corresponding account information.
7. System interacts with the **payment authorization system** to carry out the payment.
8. System informs the Primary actor that the order has been confirmed.
9. Customer acknowledges. *{Use case ends.}*

Extension Points:

*a. **Customer indicates that he/she wishes to cancel the order**
  *a1. *{Use case ends}*
4a. **Customer indicates that he/she wishes to do another product search:**
  4a1. Use case *{Use case resumes at step 1}*.
5a. **The desired product is not available in sufficient quantities.**
  5a1. System informs the customer that product is not available in desired quantity.
  5a2. *{Use case ends.}*
8a. **The payment information is invalid:**
  8a1. System informs the customer that payment information provided is invalid.
  8a2. *{Use case resumes at step 6}*

Source: https://www.researchgate.net/publication/220368776/figure/fig2/AS:305566367928326@1449864107517/Example-Use-Case-for-Order-Product.png

Use Case Example

Use Case Title: Pay for a job posting
Primary Actor: Recruiter
Level: Actor goal
Precondition: The job information has been entered but is not viewable.
Minimal Guarantees: None
Success Guarantees: Job is posted; recruiter's credit card is charged.
Main Success Scenario:

1. Recruiter submits credit card number, date, and authentication information.
2. System validates credit card.
3. System charges credit card full amount.
4. Job posting is made viewable to Job Seekers.
5. Recruiter is given a unique confirmation number.

Extensions:

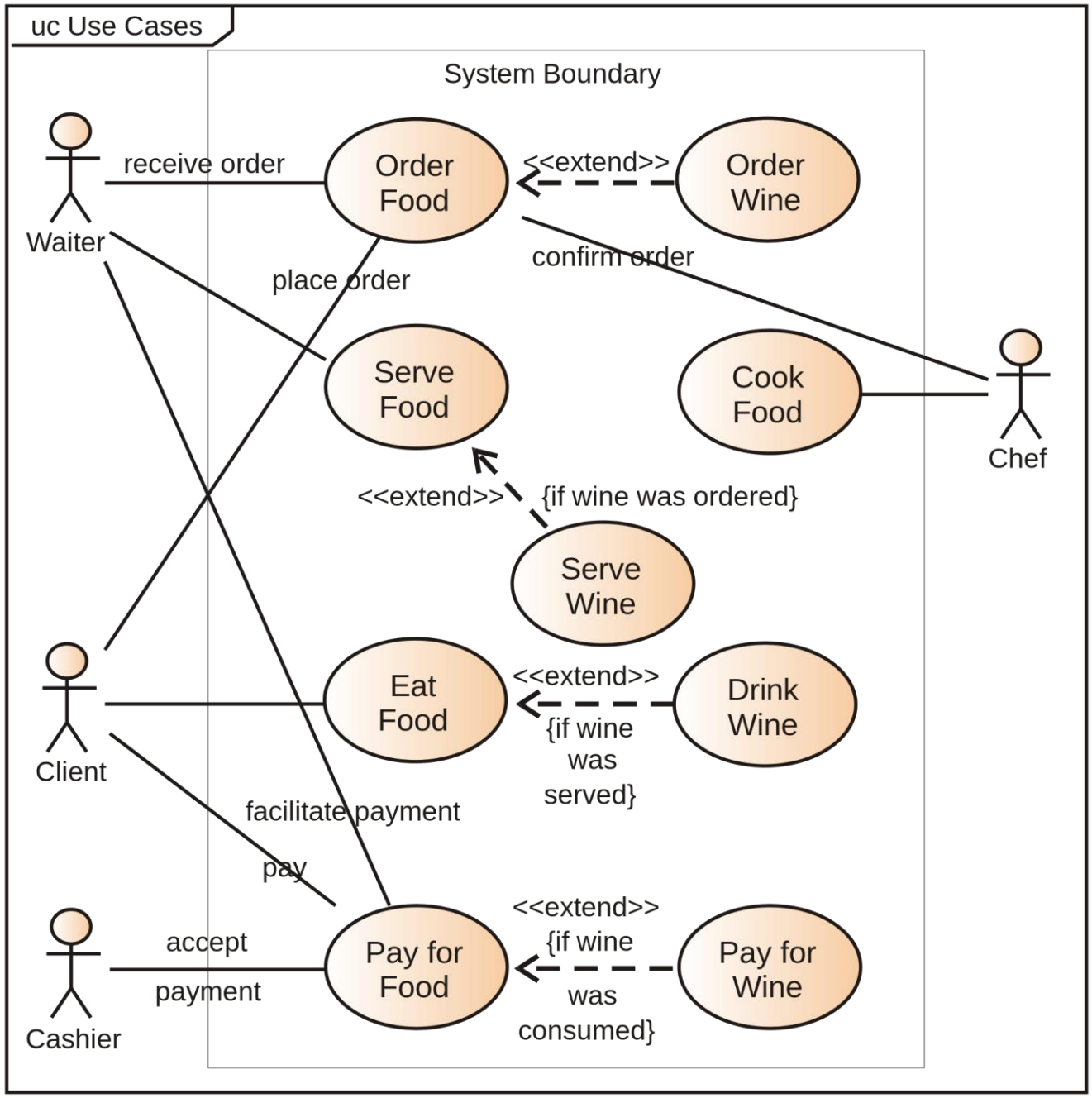
2a: The card is not of a type accepted by the system:
    2a1: The system notifies the user to use a different card.
2b: The card is expired:
    2b1: The system notifies the user to use a different card.
2c: The card is expired:
    2c1: The system notifies the user to use a different card.
3a: The card has insufficient available credit to post the ad.
    3a1: The system charges as much as it can to the current credit card.
    3a2: The user is told about the problem and asked to enter a second credit card for the remaining charge. The use case continues at Step 2.

Source: https://media.licdn.com/dms/image/C4D12AQFSqb_8arRNrg/article-inline_image-shrink_1000_1488/0/1626075696079?e=1700697600&v=beta&t=KEZb1pzTSTOGTWkETbSJbKoLnYxK65YWJ91NzjjZSsw

Use Case Diagram

- Use case diagrams can show...
  - **Actors** involves in the use case ("stick man figure")
  - **Use cases** that the system allows for (ovals)
  - **System boundary** box showing the boundary between actors and the system or parts of the system
  - **Associations** are lines associating actors with use cases
  - **Extensions** which are optional extensions to use cases
  - **Includes** which show that a use case includes another use case

Use Case Diagram Example



Source: https://en.wikipedia.org/wiki/Use_case_diagram#/media/File:Use_case_restaurant_model.svg

Use Cases vs. User Stories

- Use cases are a relatively heavier weight solution to documenting scenarios than user stories
  - Some consider user stories to be a type of use case
  - No explicit requirement for conversation with use cases
- Rigorous use cases can make explicit many things that user stories may fail to capture otherse
  - Preconditions?
  - Extensions?
    - This can take up much of development time!
- Like user stories, use cases are used focused

Use Case Drawbacks

- Fail to capture non-interaction requirements well
  - e.g. non-functional requirements
- No fully standard definition
  - Must create our own definition
- Goals are included, but what about benefits?
  - In the case of user stories, these shape what is built

Sequences

- A **sequence** is a list of interactive steps taken by users or machine agents playing system roles
  - Steps have an order
  - Sometimes called a script
- Each step is an action taken by a user or machine agent, for example...
  - Teacher selects quiz options (user)
  - Quiz preview is presented to the user (machine agent)
- Sequences somewhat translate to sequence diagrams, though sequence diagrams focus on 'back and forth' communication, timelines and synchronization more

Other Scenario Formats

- **Simulations** involve possible interactive animations of a scenario
- **Roleplays** involve acting out scenarios to better facilitate understanding
- **Flowcharts** can be used to model scenarios with different possibilities and the flow of actions that may occur

Types of scenarios

- One categorization of scenarios which does not address the format (story vs. storyboard), but rather the behavior(s) relative to the system
  - **Positive Scenario:** A scenario where the user provides valid information and follows the system's rules.
  - **Negative Scenario:** A scenario where the user provides invalid information or attempts unauthorized actions, which the system must gracefully handle (i.e. safely block and reject).
  - **Normal Scenario:** A positive scenario where everything works perfectly on the first try with no interruptions or glitches (the "happy path").
  - **Abnormal Scenario:** A positive scenario where an unexpected system error or business restriction forces the system away from the happy path to handle a problem

Positive, normal scenario example

- The customer places an order for a listed item with their selected customizations
- The customer is charged, and the payment is received by an external processor
- The order is forwarded to the kitchen ticketing system
- The order is made to customer specs by the cooks
- The customer picks up their order

- Note that the above is an example of a sequence!

Negative scenario

- The customer attempts to enter their own personal customizations
- The customer is informed only specific predefined customizations are allowed to be selected
- The customer selects from the allowable customizations
- The customer is charged, and the payment is received by an external processor
- The order is forwarded to the kitchen ticketing system with different customizations
- The order is made to ticket specs by the cooks
- The customer picks up their order

Positive, abnormal scenario

- The customer places an order with their selected customizations
- The system checks if all ingredients are in stock
- The system determines that one of the ingredients is out of stock
- The customer is notified that the order has been cancelled

Types of scenarios

- Notice how the abnormal scenario contains steps not included in the normal scenario

- Why is this?
  - The "happy path" is easiest to consider
  - Abnormal scenarios make us think about new problems that could occur

- A **happy path** is a positive, normal scenario featuring no exceptions or errors